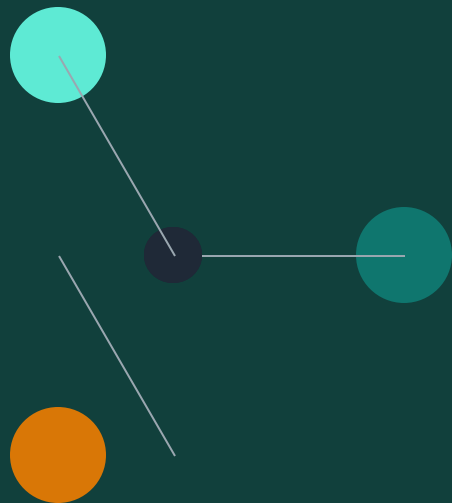


PYTHON을 이용한 데이터 분석 · 11강

클러스터링이란?

K-means의 원리

이론편 — 레이블 없이 데이터를 그룹 짓기



오늘의 학습 목표



클러스터링의 개념 이해

레이블 없이 비슷한 데이터끼리 그룹 짓는 비지도학습



K-means 알고리즘

반복적으로 중심점을 갱신해 그룹을 찾는 과정



거리와 중심점

유클리드 거리로 가까운 데이터를 같은 클러스터에 할당



최적 K 찾기

엘보우 방법과 실루엣 점수로 클러스터 개수 결정

우리는 지금 어디로 가는가?

지도학습

입력 X (특성)

정답 y (레이블 있음)

예시 9강 회귀 · 10강 분류

목표 $X \rightarrow y$ 매핑 함수 찾기

비지도학습

입력 X (특성만)

정답 없음!

예시 오늘의 클러스터링

목표 X 안에서 자연스러운 패턴 발견

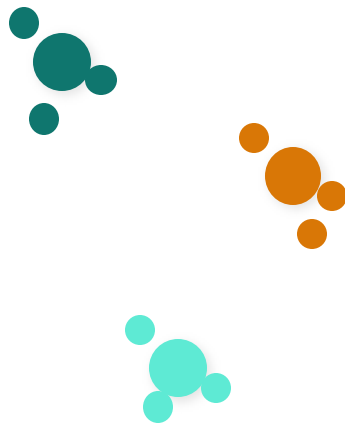
전환점 지금부터는 '정답을 외우는' 학습에서 벗어나 '패턴을 찾는' 탐험 모드로 진입합니다.

클러스터링 — 비슷한 것끼리 모으기

정의 데이터를 **레이블 없이** 비슷한 것끼리 **그룹(클러스터)**으로 나누는 비지도학습. '어떤 기준?'이라는 질문은 우리가 정의한다.

예시

- 고객 세분화 (구매 패턴별)
- 유전자 분류 (DNA 유사도별)
- 뉴스 자동 분류 (주제별)
- 이미지 압축 (색상 비슷한 픽셀)



K=3 클러스터

가장 유명한 클러스터링: K-means

K-means는 **K개의 중심점(centroid)**을 반복적으로 조정하며 클러스터를 찾는 알고리즘입니다. 스무개(결정트리)보다 훨씬 간단!

1

초기화

2

할당

3

업데이트

4

반복

K개의 중심점을 무작위로 선택 각 데이터를 가장 가까운 중심점에 배정 클러스터의 새로운 중심점 계산 중심점이 변하지 않을 때까지 2~3 반복

핵심 중심점이 안정화되는 순간까지 반복하는 것 = 수렴(convergence). 이 과정에서 클러스터가 점점 "좋아져"갑니다.

K-means 4단계 상세 설명

Step 1. 초기 중심점 선택

무작위로 K개 점 선택 (또는 K-means++ 방법 사용)

Step 2. 할당 (Assignment)

각 데이터 x 를 거리가 가장 가까운 중심점에 배정 $\rightarrow d(x, c) = \sqrt{[(x_1 - c_1)^2 + (x_2 - c_2)^2]}$

Step 3. 업데이트 (Update)

각 클러스터의 새 중심점 = 그 클러스터에 속한 모든 점의 평균 $\rightarrow c_{\text{new}} = (1/|S|)\sum_{x \in S}$

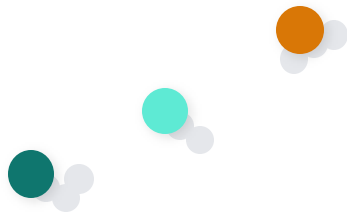
Step 4. 반복과 수렴

중심점이 변하지 않거나, 최대 반복 횟수 도달할 때까지 Step 2~3 반복

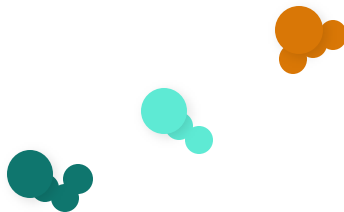
주의 K-means는 "지역 최적해"에 빠질 수 있습니다. 초기 중심점에 따라 결과가 달라질 수 있으므로, 여러 번 실행해 최고 결과를 선택합니다.

K-means가 한 번의 사이클에서 하는 일

초기 상태: 무작위 중심점 3개



할당: 각 점을 가장 가까운 중심점에



결과: 각 색깔이 하나의 클러스터 (Step 2)

다음 단계 (Step 3)에서는:

- 검은색 클러스터의 평균 → 새로운 검은색 중심
- 주황색 클러스터의 평균 → 새로운 주황색 중심
- 초록색 클러스터의 평균 → 새로운 초록색 중심

CHOOSING K

얼마나 많은 클러스터가 필요한가?

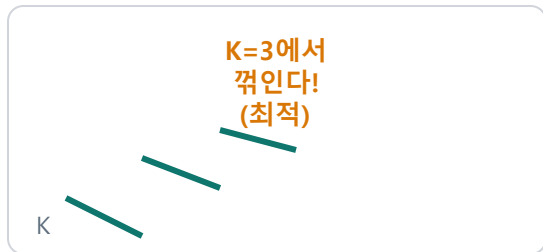
핵심 질문 K를 몇으로 설정해야 할까? 너무 작으면 서로 다른 그룹이 섞이고, 너무 크면 한 그룹이 잘못 나뉜다.

방법 1: 엘보우 (Elbow Method)

방법 2: 실루엣 점수 (Silhouette Score)

K를 1부터 늘려가며 SSE(Sum of Squared Errors)를 측정. 각 점이 자신의 클러스터에 '얼마나 잘' 속해 있는가를 -1~1로 측정. 클러스터에 '얼마나 잘' 속해 있는가. 높을수록 좋다. (elbow) 이 최적 K.

K=3이 최고!



거리를 재는 방법과 전처리의 중요성

K-means의 핵심: 거리 계산

유클리드 거리 (기본)

$$d = \sqrt{[(x_1 - c_1)^2 + (x_2 - c_2)^2]}$$

가장 직관적 (직선거리)

맨해튼 거리

$$d = |x_1 - c_1| + |x_2 - c_2|$$

격자처럼 이동 (택시 거리)

전처리의 중요성

거리는 특성의 스케일에 민감합니다. 예: 나이(0~100)와 소득(0~10,000,000)이 섞여 있으면 소득이 거리를 지배합니다.

해결책 모든 특성을 같은 스케일로 정규화(normalization)하거나 표준화(standardization). sklearn은 StandardScaler 제공.

K-means의 강점과 한계



장점 ① 빠르다

간단한 반복 계산 → 수백만 개 데이터도 처리 가능. 실시간 애플리케이션에 적합.



장점 ② 이해하기 쉽다

중심점 이동, 거리 계산 → 직관적. 결과를 비즈니스에 설명하기 쉬움.



한계 ① K 사전 결정

K를 미리 정해야 함. 엘보우나 실루엣으로 찾아야 하는 번거로움.



한계 ② 지역 최적해

초기 중심점에 따라 결과가 달라짐. 여러 번 실행 필요.



한계 ③ 원형 클러스터만

타원형이나 불규칙한 모양 클러스터를 잘 찾지 못함.

K-means 이후의 선택지

계층적 클러스터링

작은 클러스터부터 시작해 점점 합쳐가기. K를 미리 정할 필요 없음.

DBSCAN

밀도 기반 클러스터링. 원형이 아닌 불규칙한 모양도 찾음.

가우시안 혼합 모델 (GMM)

확률 기반. 각 점이 어느 클러스터에 속할 "확률"을 계산.

파이썬에서는 어떻게 할까? — 다음 강의 예고

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

model = KMeans(n_clusters=3, random_state=42)
model.fit(X_scaled)

labels = model.labels_          # 각 점의 클러스터
centers = model.cluster_centers_ # 중심점들
```

StandardScaler

특성의 스케일 맞추기 (필수!)

KMeans(n_clusters=3)

K=3으로 K-means 모델 생성

fit() / predict()

fit/predict 패턴은 9강·10강과 동일

패턴 인식 import → scaler/model → fit → predict/labels. 9강·10강과 구조가 같습니다. sklearn 숙련도가 늘어갑니다!

SUMMARY

오늘의 핵심 정리

- ✓ 비지도학습 레이블 없이 데이터의 자연스러운 그룹을 찾는 학습
- ✓ K-means 중심점을 반복해 이동하며 클러스터를 최적화하는 알고리즘
- ✓ 거리와 스케일 유클리드 거리로 가까움을 재고, 특성 스케일은 반드시 정규화
- ✓ 최적 K 결정 엘보우 방법, 실루엣 점수, 도메인 지식을 활용
- ✓ 장단점 인식 빠르고 간단하지만, K를 미리 정해야 하고 원형 클러스터만 잘 찾음

NEXT 11강 실습 — 고객 세분화(customer segmentation) 데이터로 K-means 구현